

Minimal Resolver Configuration

In this simulated deployment, a minimal resolver (MicroMutableOpResolver) was implemented instead of an all-inclusive operator resolver to strictly minimize the runtime memory footprint. The 8 specific operators required for inference were identified in two stages. First, the core neural network operations (Conv2D, MaxPool2D, FullyConnected, Reshape, Softmax) were directly mapped from the original Keras model architecture. Second, the implicit operators introduced automatically during the INT8 quantization conversion (Shape, StridedSlice, Pack) were identified through iterative hardware debugging. This was achieved by executing the binary and analyzing the terminal error logs whenever a missing operator caused a runtime memory allocation failure.

Memory Optimization

Eliminating the overhead of unused operators through the minimal resolver enabled extreme compression of the Tensor Arena. To determine the absolute minimum memory requirement for the system, an empirical profiling and boundary testing methodology was employed. Initially, a safely large arena size (60 KB) was allocated to ensure successful execution. By analyzing the framework's runtime diagnostic output, the actual memory footprint was identified as exactly 8,976 bytes. The kTensorArenaSize was then adjusted to this precise value to confirm stability. Finally, a strict boundary test was conducted by decrementing the size by a single byte (8,975 bytes), which intentionally triggered an AllocateTensors() failure and a segmentation fault, conclusively proving that 8,976 bytes is the strict minimum threshold required for this model.

kTensorArenaSize = 60 * 1024:

```
--- Tensor Metadata (Sanity Check) ---
Input Tensor - Type: 9, Byte Size: 784, Scale: 0.003922, Zero Point: -128
Output Tensor - Type: 9, Byte Size: 10, Scale: 0.146342, Zero Point: 5

--- Memory Usage Analysis ---
Model Size (Bytes): 26560
Total Arena Size: 61440
Arena Used: 8976 bytes

--- Running Inference ---
Test Input Result -> Predicted Class: 1
```

kTensorArenaSize = 9 * 1024

```
--- Tensor Metadata (Sanity Check) ---  
Input Tensor - Type: 9, Byte Size: 784, Scale: 0.003922, Zero Point: -128  
Output Tensor - Type: 9, Byte Size: 10, Scale: 0.146342, Zero Point: 5  
  
--- Memory Usage Analysis ---  
Model Size (Bytes): 26560  
Total Arena Size: 9216  
Arena Used: 8976 bytes  
  
--- Running Inference ---  
Test Input Result -> Predicted Class: 1
```

kTensorArenaSize = 8976

```
--- Tensor Metadata (Sanity Check) ---  
Input Tensor - Type: 9, Byte Size: 784, Scale: 0.003922, Zero Point: -128  
Output Tensor - Type: 9, Byte Size: 10, Scale: 0.146342, Zero Point: 5  
  
--- Memory Usage Analysis ---  
Model Size (Bytes): 26560  
Total Arena Size: 8976  
Arena Used: 8976 bytes  
  
--- Running Inference ---  
Test Input Result -> Predicted Class: 1
```

kTensorArenaSize = 8975

```
Failed to allocate tail memory. Requested: 24, available 16, missing: 8  
Unable to allocate TfLiteAffineQuantization.  
  
Failed to populate a persistent TfLiteTensor struct from flatbuffer data!  
Failed to initialize output tensor 0  
AllocateTensors() failed  
  
--- Memory Usage Analysis ---  
Model Size (Bytes): 26560  
Total Arena Size: 8975  
Arena Used: 8959 bytes  
Segmentation fault ./micro_inference_app
```